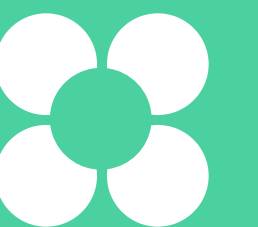


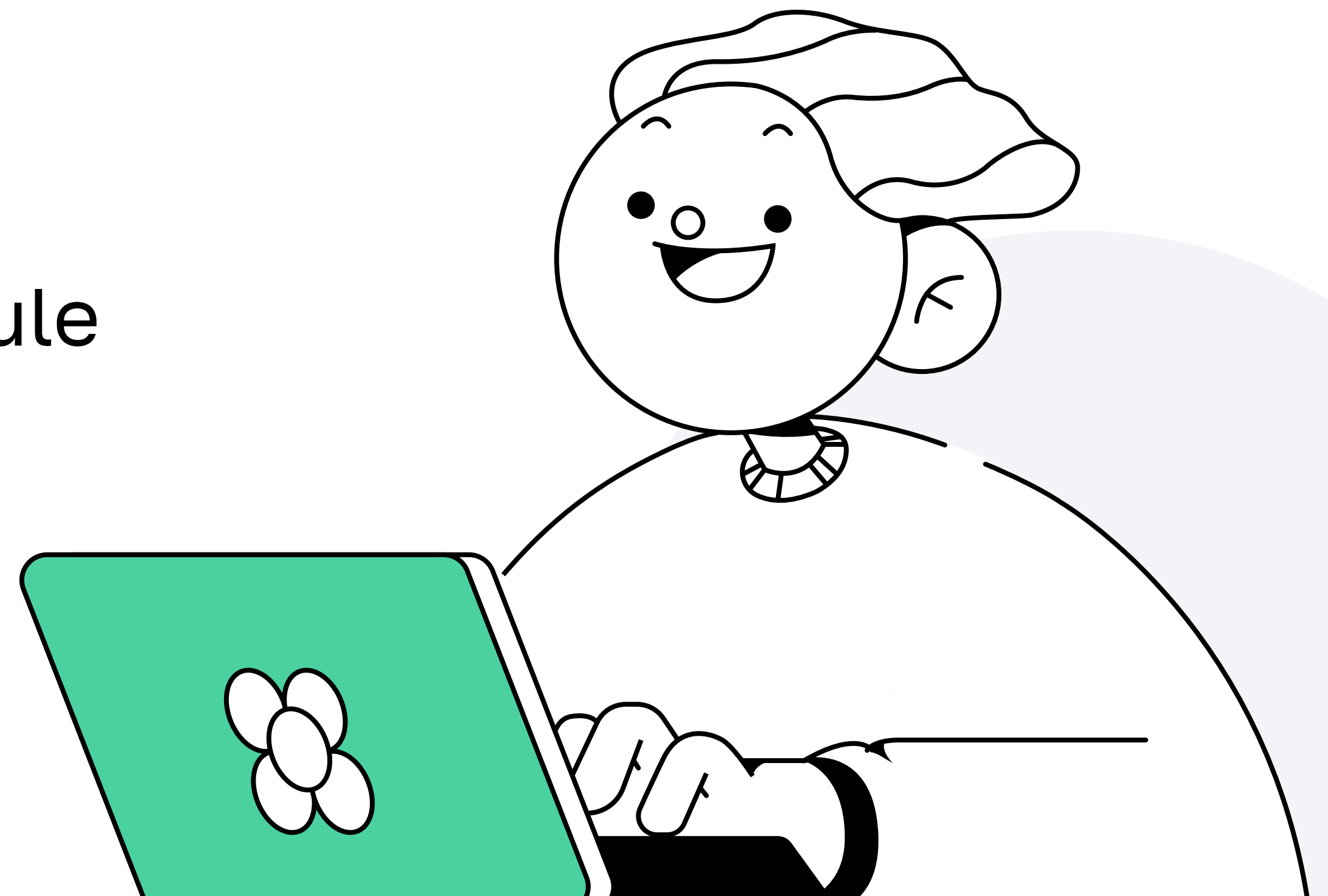
# Создание собственных modules

Алексей Метляков  
Tech lead DevOps engineer в OpenWay



# План занятия

- 1 Plugin & module
- 2 Исполнение module в Ansible
- 3 Совместимость Python 2 и Python 3
- 4 Реализация module
- 5 Хранение и использование module



# Plugin & module

Алексей Метляков

Tech lead DevOps engineer в OpenWay



# Что такое plugin

**Plugin** – спроектированный на **Python** код, который выполняет одно логическое действие





# Типы plugin

Все **plugin** делятся на несколько типов:

- connection
- become
- inventory
- vars
- shell
- lookup
- strategy
- modules

# Что такое module

**Module** — написанный на языке программирования программный код, основная цель которого — реализовать манипуляции над входными данными, обработать эти данные на стороне исполнителя и отправить выходные данные в **JSON**-формате



# Что такое module

**Module** должен:

- принять данные на вход
- обработать их
- провести действия в системе
- получить результат этих действий
- подготовить выходные данные из сырого формата в json
- отправить ответ в сторону Ansible



# Что такое module

Вызвать **module** для использования можно двумя путями:

✓ через командную строку:

```
ansible localhost -m service -a "name=memcached state=restarted"
```

✓ описать в плейбуке:

```
---
- name: Restart memcached
  service:
    name: memcached
    state: restarted
...
```

# Что такое module

Справку по любому module можно получить через команду:

```
ansible-doc
```

# Исполнение module в Ansible

Алексей Метляков

Tech lead DevOps engineer в OpenWay



# Как Ansible исполняет module

- ➔ Пользователь указывает в **task** имя **module** и его параметры
- ➔ **TaskExecutor** получает на вход имя **module**, определяет его суть (**module** или **action plugin**), передаёт в **normal action plugin** полученную информацию
- ➔ **Normal action plugin** выполняет основную работу на удалённом хосте, он основной её координатор:
  - вызывает **connection plugin**, чтобы установить соединение
  - добавляет внутренние свойства **Ansible** к **module**
  - работает с остальными **plugin**, чтобы создать временные файлы
  - отправляет **module** и параметры на удалённый хост
  - обрабатывает любые исключительные случаи работы модулей

# Как Ansible исполняет module

- ➔ Запускается **Executor/module\_common.py**. Он собирает **module**, чтобы отправить на удалённый узел, предварительно определив его тип:
  - **PowerShell** и **JSON-args modules** передаются через **Module Replacer**
  - **New-style Python modules** собирает **Ansiballz framework**
  - **Non-native-want-JSON**, **Binary** и **Old-style modules** отправляются на удалённый узел без сборки в том виде, в котором они есть
- ➔ После подготовки проверяется директива **shebang** и сравнивается с тем, что, возможно, указано в параметрах **Ansible**. Если в **Ansible** отличается, подставляет значение **module**
- \* Полное описание типов module



# Как Ansible исполняет module

→ В нашем случае module собирается при помощи **Ansiballz framework**:

- узнаёт, какие зависимости есть у **module**
- создаёт **zip** с **module** и его зависимостями и шаблоном передачи параметров **module**
- **zip** кодируется **base64**, упаковывается в небольшой **python script** и отправляет на удалённый хост во временное хранилище
- извлекает только код **module** в это же хранилище
- устанавливает **PYTHONPATH** на **zip** для поиска зависимостей
- импортирует **module** как **\_\_main\_\_**, чтобы была возможность исполнять **python wrapper** и код **module** в одном процессе

# Как Ansible исполняет module

- В конце исполнения **module** формирует **JSON**-строку с результатом и отправляет её на стандартный вывод;
- The **normal action plugin** десериализует этот **JSON** в **dict** и возвращает в **executor**
- **Jinja**-шаблоны будут восприняты **Ansible** как обычные строки, если они помечены как небезопасные
  - Строки, возвращаемые через **ActionPlugin.\_execute\_module()**, автоматически помечаются как небезопасные через **normal action plugin**
  - В остальных случаях строки необходимо пометить самостоятельно

# Совместимость Python 2 и Python 3

Алексей Метляков

Tech lead DevOps engineer в OpenWay



# Совместимость Python 2 и Python 3

- На стороне **control node ansible** поддерживает **Python**-версии  **$\geq 3.5$**  и  **$\geq 2.7$**
- На стороне **managed node ansible** поддерживает **Python**-версии  **$\geq 3.5$**  и  **$\geq 2.6$**
- **Python  $\geq 3.5$**  выбран, так как используется в **LTS**-дистрибутивах
- **Python 2.6** может работать не со всеми **modules**, например, **docker-py**

# Совместимость Python 2 и Python 3

Разработчики **Ansible** предлагают воспользоваться [guide](#) о сохранении совместимости между разными версиями **Python**. Сами разработчики используют [единую базу](#) кода.

- В Python 3 строка может быть массивом байтов, как в C, или массивом слов
- Чтобы объединять эти типы строк, их нужно предварительно преобразовывать друг к другу
- Python 2 использует эти виды как взаимозаменяемые (str для байтов и unicode для строк). Строки автоматически преобразуются, пока состоят из ASCII, иначе вызывается исключение

# Совместимость Python 2 и Python 3

**Unicode sandwich** – стратегия, при которой строки вне работы модуля используются как массив байтов, а внутри обрабатываются как массив слов





# Совместимость Python 2 и Python 3

Чтобы преобразовать данные в байты или слова, есть два импорта:

```
from ansible.module_utils._text import to_bytes  
from ansible.module_utils._text import to_text
```

# Совместимость Python 2 и Python 3

→ Использовать шаблон прямой совместимости:

```
from __future__ import (absolute_import, division, print_function)
__metaclass__ = type
```

→ `__metaclass__ = type` — превращает все классы в классы нового стиля

→ `__future__` делает следующее:

- **absolute\_import** — заставляет импорт искать зависимости в **sys.path**, игнорируя директорию с **module**
- **division** — заставляет при делении всегда возвращать число с плавающей запятой, если нужно получить частное - `x // y`
- **print\_function** — оборачивает стандартный вывод в отдельную функцию



# Реализация module

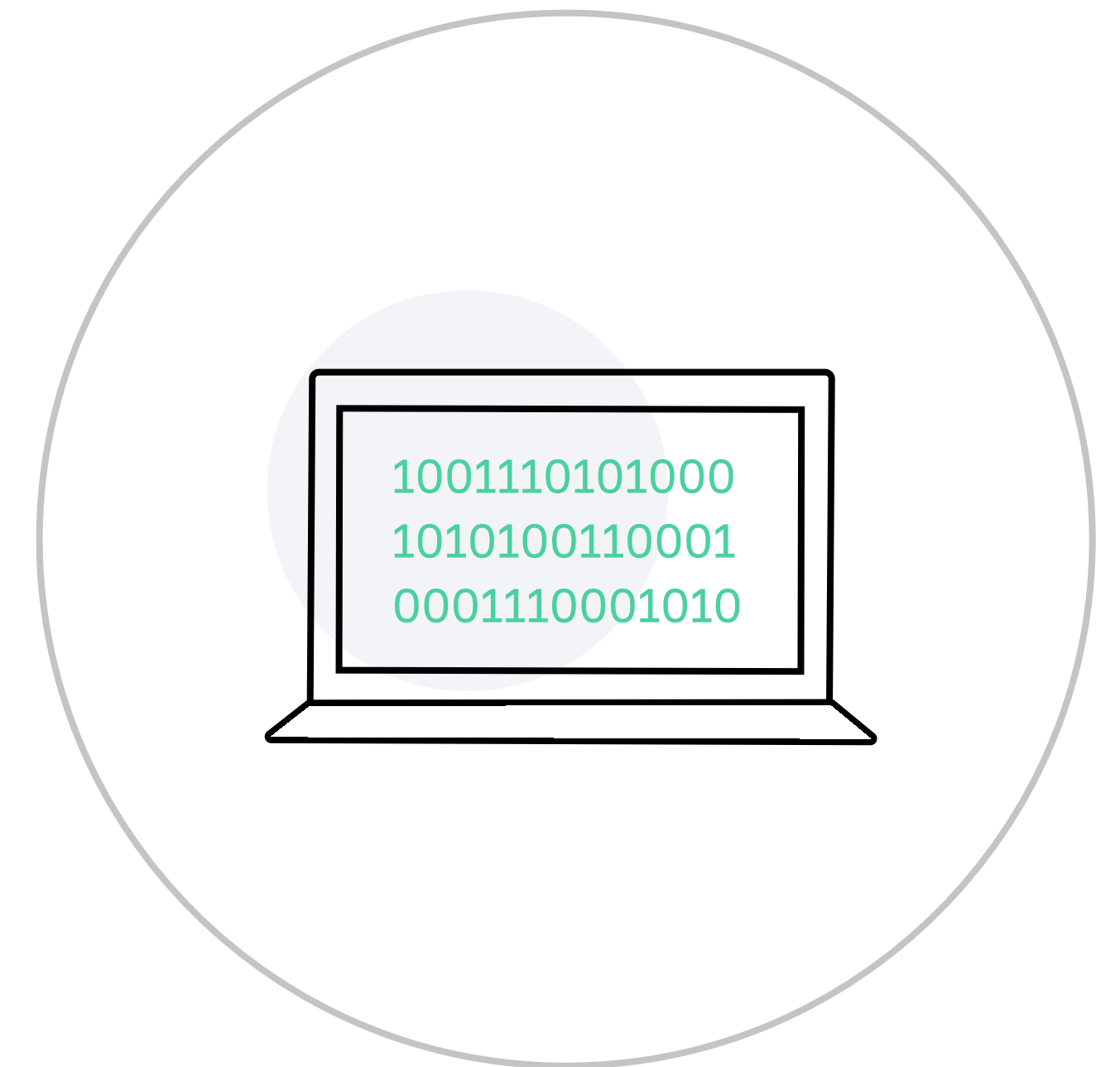
Алексей Метляков

Tech lead DevOps engineer в OpenWay

# Когда стоит писать свой module

Прежде чем вы начнёте писать собственный **module**, документация предлагает ответить на четыре вопроса:

- существует ли похожий **module**
- стоит ли использовать **module** вместо **action plugin**
- можно ли написать **role** вместо **module**
- стоит ли создать **collection** вместо **module**



# Когда стоит писать свой **module**

**Module** должен соответствовать требованиям:

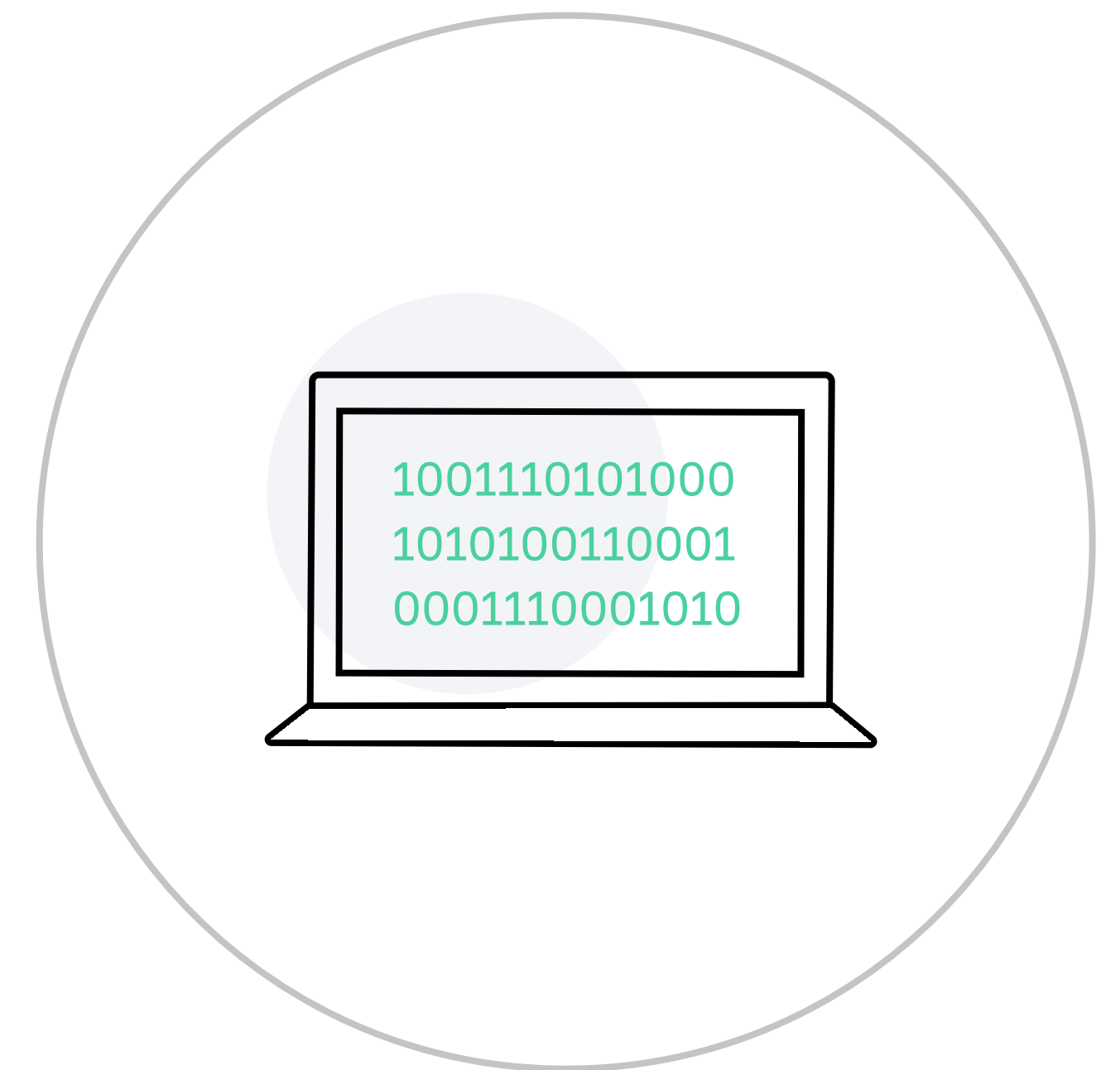
- Каждый **module** должен иметь атомарную функциональность и преследовать **UNIX-философию** — делать всё хорошо
- **Module не должен** требовать глубоких познаний в той области, в которой он работает. Например, если необходимые значения обязательных параметров не могут быть задокументированы, **module** лучше отклонить
- **Module должен** обладать достаточно ёмкой логикой. Например, если он работает с API-инструмента и просто отдаёт информацию пользователю, при его использовании будет слишком много логики внутри **playbook**



# Простейший module

Взглянем на структуру самого простого **module**:

- указание пути до исполняемого бинарника через **shebang**
- отдельная переменная с документацией
- переменная с примерами использования
- переменная с возможными ответами **module**
- исполняемый код записан в функцию **main()**
- функция **main()** вызывается при старте исполнения **module**



# От идеи к реализации

Основные советы к реализации:

- Один **module** должен содержаться в одном файле
- Название **module** нужно разделять подчёркиванием, использование дефиса или пробела недопустимо
- Всегда используйте **ansible-test**
- Старайтесь минимизировать зависимости
- Не производите запись в файлы напрямую. Лучше сделать записи во временный файл, а потом перенести их при помощи **atomic\_move** функции из **ansible.module\_utils.basic**
- Избегайте создания кешей

# От идеи к реализации

## Основные советы к реализации:

- Каждая функция должна именоваться с использованием подчёркивания в качестве разделителя
- «Не повторяйтесь» — это лучшая философия
- Функции должны быть лаконичными и описывать большой объём работы
- Имя функции в полном объёме обозначает её назначение
- Каждая функция должна иметь строку с описанием
- У кода должна быть минимальная вложенность. Если вложенность большая, лучше переиспользовать его в функциях



# От идеи к реализации

Основные советы к реализации:

- При обработке **URL** лучше использовать **fetch\_url** или **open\_url** из **ansible.module\_utils.urls** вместо **urllib2**
- Используйте **main()** для оборачивания основного кода **module**
- Вызывайте **main()** через условие, чтобы можно было импортировать её в тесты

# От идеи к реализации

- Для начала установите зависимости, например, для Ansible: **build-essential, libssl-dev, libffi-dev, python-dev**
- Клонировать репозиторий [Ansible](#) и войдите в его директорию
- Создайте виртуальное окружение: **python3 -m venv venv**
- Активируйте виртуальное окружение **. venv/bin/activate**
- Установите зависимости через **pip**
- Запускайте **env-setup** скрипт каждый раз, когда создаёте новый **shell**-процесс: **. hacking/env-setup**

Теперь окружение готово к работе. Каждый раз при желании начать работу нужно выполнять: **. venv/bin/activate && . hacking/env-setup**



# От идеи к реализации

- Перейдите в `lib/ansible/modules`
- Создайте новый `.py` файл для модуля
- Заполните его содержимым
- Исправьте содержимое так, чтобы **module** выполнял нужную вам функцию, в соответствии с рекомендациями **Ansible**
- Если ваш module работает только с локальным хостом:
  - Создайте JSON-файл с наполнением аргументов
  - Запустите виртуальное окружение
  - Запустите свой module: `python -m ansible.modules.<module_name> <path_to_json>`

# От идеи к реализации

- Дополнительно рекомендуется использовать **ansible-test**, чтобы тестировать **modules**
- Можно использовать дополнительные **framework** для тестирования, например, **pytest**
- Для полного цикла тестирования нужно создать простейший **playbook** из одной **task**, которая вызывает этот **module** и проверяет его работоспособность

Если все указанные пункты пройдены, вы справились, и ваш **module** готов отправляться в свободное плавание

# Хранение и использование module

Алексей Метляков

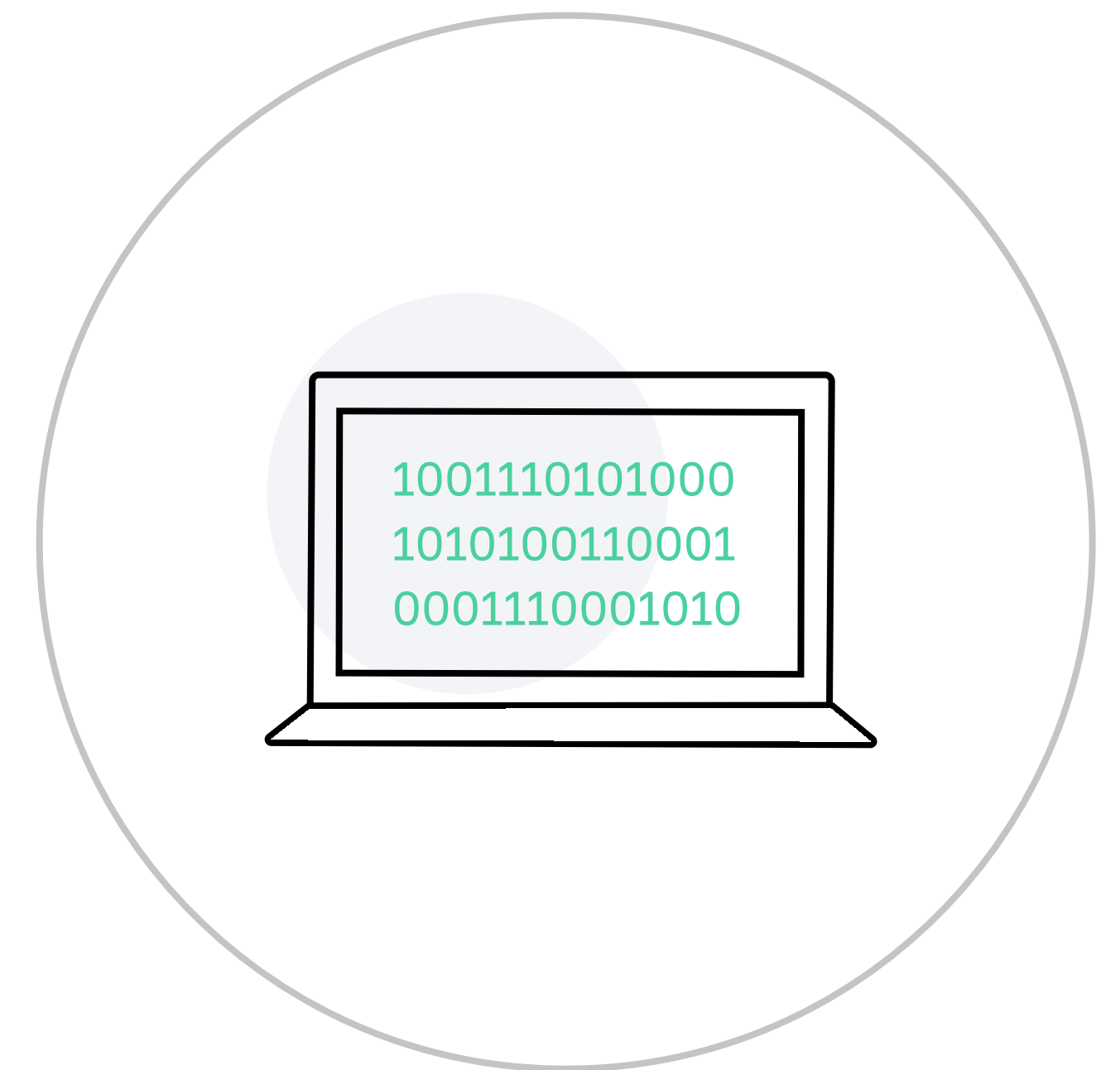
Tech lead DevOps engineer в OpenWay



# Хранение и использование module

Места хранения module определены самим Ansible:

- может находиться в системной директории **Ansible**
- рядом с **role** в директории **plugins**
- в **collection** в директории **plugins**



# Ansible Collections

**Collection** — способ доставки **Ansible**-артефактов, структурно может содержать:

- **roles**
- **playbooks**
- **modules**
- **tests**



# Ansible Collections

- Именованье collection подчиняется следующему правилу:  
**<namespace>.<collection>**
- Чтобы создать пустую коллекцию, нужно воспользоваться конструктором: **ansible-galaxy collection init <namespace>.<collection>**



# Ansible Collections

**Collection** можно сделать общедоступными, но так как мы рассматриваем частный случай использования:

- **Collection** можно отправить на хранение в свой частный репозиторий
- Чтобы скачать collection на другой хост, нужно воспользоваться:  
**ansible-galaxy collection install -r requirements.yml** со следующим содержимым:

```
---  
collections:  
  - name: <url_to_repo>  
type: git  
version: <version_tag>
```

- **Collection** можно локально собрать в **.tar.gz** и передавать в таком виде:  
**ansible-galaxy collection build**